Puppet LVM Resize Module

RHEL 8 / 9 / 10 - custom OpenVox/Puppet module documentation

Purpose: Grow configured LVM-backed filesystems when their current Facter size is below the desired target.

Module layout

```
custom/
  manifests/lvm_resize.pp
  functions/source_candidates.pp
  files/lvm_resize/check_lvm_resize.sh
  files/lvm_resize/resize_lvm.sh
```

Configuration

```
include custom::lvm_resize

custom::lvm_resize::partitions:
  - mount: /
    size: 24G
  - mount: /var
    size: 24G
```

Behavior and safety: Facter performs the catalog-level size gate. Immediately before resizing, a live check verifies that every mount is LVM-backed, every logical volume shares one volume group, and the group can satisfy all required growth. lvextend --resizefs grows the logical volume and filesystem together. The module never shrinks storage.

Supported platform: Red Hat Enterprise Linux 8, 9, and 10. Generated from the repository sources; rerun **tools/generate_custom_module_docs.ps1** after code changes.

LVM Resize

custom/manifests/lvm_resize.pp

```
# @summary
#   Grows configured LVM-backed filesystems to their desired sizes.
#
# @param partitions
#   Mount points and target sizes. Sizes are whole gibibytes such as 24G.
class custom::lvm_resize (
  Array[Struct[{
    'mount' => Pattern[/^\/[A-Za-z0-9._\/-]*$/],
    'size'  => Pattern[/^[1-9][0-9]*[Gg]$/],
  }], 1] $partitions = [
    {
      'mount' => '/',
      'size'  => '24G',
    },
    {
      'mount' => '/var',
      'size'  => '24G',
    },
  ],
) {
  if $facts['os']['family'] != 'RedHat' {
    fail('custom::lvm_resize supports only Red Hat family operating systems')
  }

  case $facts['os']['release']['major'] {
    '8', '9', '10': { }
    default: {
      fail('custom::lvm_resize supports RHEL major versions 8, 9, and 10 only')
    }
  }

  $partitions.each |Struct $partition| {
    $mount = $partition['mount']
    if !($mount in $facts['mountpoints']) {
      fail("custom::lvm_resize cannot find mount point '${mount}' in the mountpoints fact")
    }
  }

  file { 'custom LVM resize check script':
    ensure       => file,
    path         => '/usr/local/sbin/check_lvm_resize.sh',
    owner        => 'root',
    group        => 'root',
    mode         => '0750',
    source       => custom::source_candidates('puppet:///modules/custom/lvm_resize/check_lvm_resize.sh'),
    sourceselect => 'first',
  }

  file { 'custom LVM resize script':
    ensure       => file,
    path         => '/usr/local/sbin/resize_lvm.sh',
    owner        => 'root',
    group        => 'root',
    mode         => '0750',
    source       => custom::source_candidates('puppet:///modules/custom/lvm_resize/resize_lvm.sh'),
    sourceselect => 'first',
  }

  $partitions_needing_resize = $partitions.filter |Struct $partition| {
    $mount = $partition['mount']
    $target_gib = Integer(regsubst($partition['size'], '[Gg]$', ''))
    $target_bytes = $target_gib * 1024 * 1024 * 1024
    $facts['mountpoints'][$mount]['size_bytes'] < $target_bytes
```

```
  }

  $partition_arguments = $partitions.map |Struct $partition| {
    "${partition['mount']} ${partition['size']}"
  }.join(' ')

  if !empty($partitions_needing_resize) {
    exec { 'resize custom LVM partitions':
      command   => "/usr/local/sbin/resize_lvm.sh ${partition_arguments}",
      onlyif    => "/usr/local/sbin/check_lvm_resize.sh ${partition_arguments}",
      path      => ['/usr/bin', '/usr/sbin', '/bin', '/sbin'],
      timeout   => 0,
      logoutput => true,
      require   => [
        File['custom LVM resize check script'],
        File['custom LVM resize script'],
      ],
    }
  }
}
```

```
# @summary
#   Returns ordered source candidates for a custom module file.
#
# @param default_source
#   Default puppet:/// source URI. Site-specific files use suffixes beside it.
function custom::source_candidates(String[1] $default_source) >> Array[String] {
  $hostname = $facts['networking']['hostname']
  $network = $trusted['extensions']['pp_network']
  $project = $trusted['extensions']['pp_project']
  $role = $trusted['extensions']['pp_role']

  $hostname_sources = $hostname ? {
    undef   => [],
    ''      => [],
    default => ["${default_source}.${hostname}"],
  }
  $network_sources = $network ? {
    undef   => [],
    ''      => [],
    default => ["${default_source}.${network}"],
  }
  $project_sources = $project ? {
    undef   => [],
    ''      => [],
    default => ["${default_source}.${project}"],
  }
  $role_sources = $role ? {
    undef   => [],
    ''      => [],
    default => ["${default_source}.${role}"],
  }

  $hostname_sources +
    $network_sources +
    $project_sources +
    $role_sources +
    [$default_source]
}
```

```
#!/bin/bash

set -u
export LC_ALL=C

if (( $# == 0 || $# % 2 != 0 )); then
    echo "Usage: $0 <mount> <target> [<mount> <target> ...]" >&2
    exit 1
fi

total_needed=0
vg_name=""

while (( $# > 0 )); do
    mount_point="$1"
    target="$2"
    shift 2

    device=$(findmnt -n -o SOURCE --target "$mount_point") || {
        echo "Unable to determine device for $mount_point" >&2
        exit 1
    }

    lvs "$device" >/dev/null 2>&1 || {
        echo "$mount_point is not an LVM logical volume: $device" >&2
        exit 1
    }

    current_vg=$(lvs --noheadings -o vg_name "$device" | xargs)
    if [[ -z "$current_vg" ]]; then
        echo "Unable to determine volume group for $device" >&2
        exit 1
    fi

    if [[ -z "$vg_name" ]]; then
        vg_name="$current_vg"
    elif [[ "$vg_name" != "$current_vg" ]]; then
        echo "Configured logical volumes are not in the same volume group" >&2
        exit 1
    fi

    current_bytes=$(lvs --noheadings --units b --nosuffix -o lv_size "$device" | xargs | cut -d. -f1)
    target_number="${target%[Gg]}"
    if ! [[ "$target" =~ ^[1-9][0-9]*[Gg]$ ]]; then
        echo "Invalid target size: $target" >&2
        exit 1
    fi
    target_bytes=$((target_number * 1024 * 1024 * 1024))

    if (( current_bytes < target_bytes )); then
        needed=$((target_bytes - current_bytes))
        total_needed=$((total_needed + needed))
        echo "$mount_point needs $needed additional bytes"
    fi
done

if (( total_needed == 0 )); then
    exit 1
fi

vg_free=$(vgs --noheadings --units b --nosuffix -o vg_free "$vg_name" | xargs | cut -d. -f1)
if [[ -z "$vg_free" ]]; then
    echo "Unable to determine free space in volume group $vg_name" >&2
```

```
    exit 1
fi

if (( vg_free < total_needed )); then
    echo "Insufficient free space in volume group $vg_name" >&2
    echo "Required: $total_needed bytes" >&2
    echo "Available: $vg_free bytes" >&2
    exit 1
fi

exit 0
```

LVM Resize

custom/files/lvm_resize/resize_lvm.sh

```
#!/bin/bash

set -euo pipefail
export LC_ALL=C

if (( $# == 0 || $# % 2 != 0 )); then
    echo "Usage: $0 <mount> <target> [<mount> <target> ...]" >&2
    exit 1
fi

while (( $# > 0 )); do
    mount_point="$1"
    target="$2"
    shift 2

    if ! [[ "$target" =~ ^[1-9][0-9]*[Gg]$ ]]; then
        echo "Invalid target size: $target" >&2
        exit 1
    fi

    device=$(findmnt -n -o SOURCE --target "$mount_point")
    lvs "$device" >/dev/null 2>&1 || {
        echo "$mount_point is not an LVM logical volume: $device" >&2
        exit 1
    }

    current_bytes=$(lvs --noheadings --units b --nosuffix -o lv_size "$device" | xargs | cut -d. -f1)
    target_number="${target%[Gg]}"
    target_bytes=$((target_number * 1024 * 1024 * 1024))

    if (( current_bytes < target_bytes )); then
        echo "Extending $mount_point ($device) to $target"
        lvextend --resizefs -L "$target" "$device"
    else
        echo "$mount_point is already $target or larger"
    fi
done
```